char** p 自增后指向了哪里？二级指针步长陷阱

Benjamin　2026-10-11　公众号文章 · 原链接 mp.weixin.qq.com

先看一段代码。这是一道流传多年的二级指针面试题：`ptr++` 执行之后，最后一行输出什么？

```c
#include <stdio.h>

int main(void)
{
    char* str[] = {"abc", "def", "ghi"};
    char** ptr = str;
    ptr++;
    printf("%s\n", *ptr);
    return 0;
}
```

我见过两个高频答案：`"bc"` 和 `"abc"`。都错。正确答案是 `"def"`，而且把这条推导走通，多级指针的全部算术就一次讲清了。

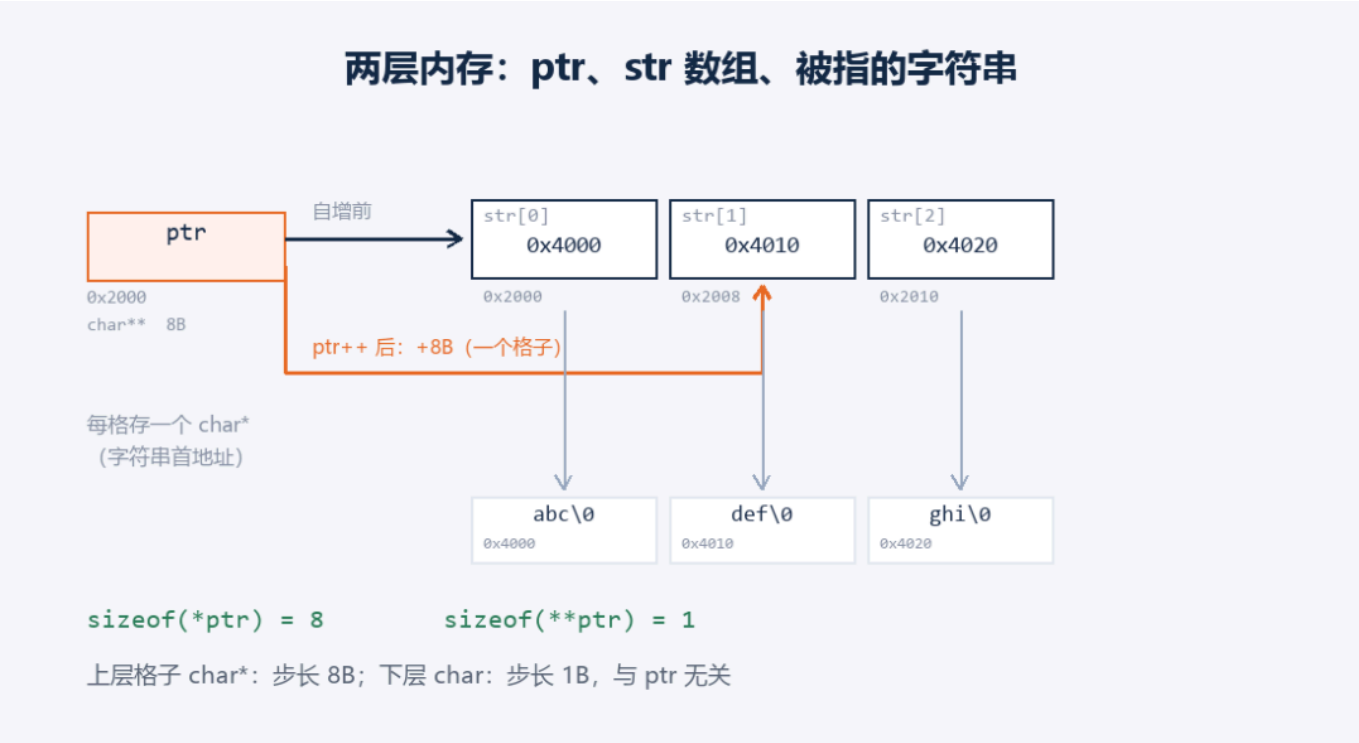
一、两个流行推演，错在哪

推演甲（答 `"bc"`）：把 `ptr` 当成 `char *` 用。既然 `ptr` 指向字符串 `"abc"`，自增一步就是往前挪一个字符（1 字节），挪过 `'a'` 之后剩下的自然就是 `"bc"`。这个推演步步顺滑，错的只有起点：`ptr` 指向的根本不是字符串，而是装字符串地址的那个数组元素。

推演乙（答 `"abc"`）：知道 `ptr` 是二级指针，但说不清自增到底动了什么，凭感觉认为"还在原来的字符串里"。它比推演甲诚实，错因相同：没建立"`ptr++` 的一次跨越，跨的是哪个层级的东西"这个画面。

正确推演只要一句：`str` 是指针数组，三个元素各存一个 `char *`；`ptr` 指向 `str[0]`；`ptr++` 按"所指向类型"的大小跨一步，所指向类型是 `char *`，64 位下 8 字节，正好从一个格子落到下一个格子。于是 `*ptr` 是 `str[1]`，即 `"def"` 的首地址，`printf("%s")` 输出 `def`。

顺带补一个背景：`char **ptr = str;` 这行赋值为什么合法？用的正是篇 1 讲过的退化规则。`str` 作为数组名，在表达式中退化为指向首元素的指针，首元素类型是 `char *`，退化产物就是 `char **`。退化发生在赋值那一刻，之后的 `ptr++` 就是纯粹的指针算术了。



图：`char *str[3]` 与 `char **ptr` 的两层内存：上层是装地址的数组，下层是被指的字符串

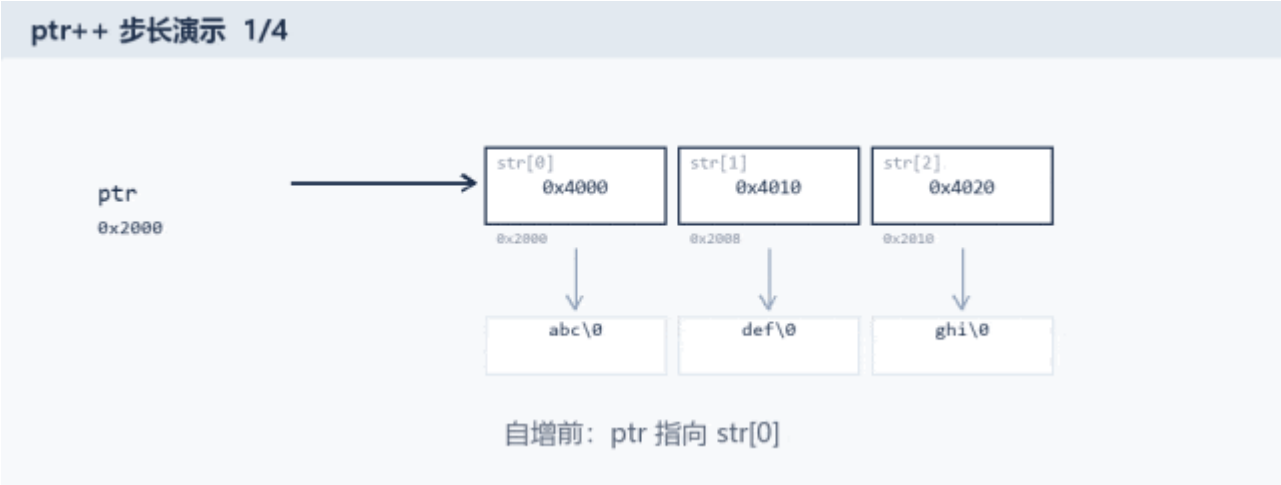
二、步长铁律：只看"所指向的类型"

用上一篇的四要素框架拆 `ptr`，四个问题四个答案：

要素	对 ptr 的答案	决定什么
指针的类型	`char **`	能赋值给谁
所指向的类型	`char *`	步长（8 字节）与解引用结果
指针的值	`&str[0]`	解引用访问哪里
自身占的内存	8 字节	与"几级"无关

指针算术只有一条规则：`p ± n` 移动 `n × sizeof(所指向的类型)` 字节。这条规则写在标准里，C11 §6.5.6 对指针加减整数的定义就是"按所指向类型的尺度缩放"。对 `char **ptr`，所指向的类型是 `char *`（8 字节），所以 `ptr++` 是 +8；最内层的 `char` 不参与计算，"再往里一层是什么"跟步长毫无关系。

推演甲的错误由此定罪：它拿 `char`（1 字节）当步长，用错了层级。推演乙则压根没有"`ptr` 指向 `str[0]`"这张图。动画把两种走法摆在一起看：



图：`ptr++` 的步长动画：误区 +1 字节 vs 正解 +8 字节，`*ptr` 最终得到 "def"

两级解引用各自得到什么，也值得单独走读一遍。`*ptr` 读的是上层格子：从 `str[1]` 里取出一个 `char *` 值，即 `0x4010`；`**ptr` 拿这个地址去下层读一个 `char`，得到 `'d'`。`printf` 的 `%s` 从 `*ptr` 开始按字符串打印，所以输出 `def`；若写 `printf("%c", **ptr)`，输出的就是单个字符 `d`。两层各读各的存储：第一次解引用跨的是 8 字节的指针格子，第二次跨的是字符串里的字节。这段走读顺带回答一个常见追问：`sizeof(*ptr)` 是多少？答案 8 而不是 1，因为 `*ptr` 的类型是 `char *`；`sizeof(**ptr)` 才是 1。

外推到任意层级，规则不变：

声明	所指向的类型	自增一步跨越
`int *p`	`int`	4 字节
`char **p`	`char *`	8 字节
`int ***p`	`int **`	8 字节

注意第二行和第三行：步长不是"指针占 8 字节"就恒为 8，`int *p` 的步长就是 4。把"指针自身大小"和"步长"混为一谈，是紧挨着本篇陷阱的又一个高频错误。

三、工程现场：argv 与二维数组

`char **` 不是面试官的玩具，`main` 的第二个参数就是它：

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    for (char **p = argv; *p != NULL; p++)
        printf("[%s]\n", *p);
    return 0;
}
```

`argv` 是"参数指针数组"的首地址，`p++` 每次跨一个 `char *`，`*p` 依次是每个参数的地址。C 标准保证 `argv[argc]` 是 `NULL`，这个循环于是成了遍历命令行最干净的写法。内核里解析模块参数、环境变量，用的都是同一模型；环境变量数组 `envp` 也是同构的 `char **`，`getenv` 的便利背后就是这层结构。

同一条铁律还能拦下一类真实 bug：把二维数组名塞给二级指针。

```c
int a[3][4];
int **p = (int **)a;    // 编译能过（强转），运行大概率炸
p[1][2];                // 把 a[0][2]、a[0][3] 拼成的值当地址解引用
```

`a` 退化后的类型是 `int (*)[4]`，步长 16 字节，与上一篇的 `int (*)[3]` 同族；`int **` 的步长是 8，并且它认定 `*p` 里存的是地址。强转骗过了编译器：`p[1]` 偏移8字节，读到的是 `a[0][2]`（低 32 位）与 a[0][3]（高 32 位）拼成的整数值，并把它当成地址，`p[1][2]` 再对这个假地址解引用，大概率当场段错误。正确的形参只有一种写法：`int (*p)[4]`。踩过这个坑的人，才会真正记住"步长与层级各管各的"。

给一个防坑检查法。写二级指针形参之前先问一句：我手里这串内存，是"装地址的数组"还是"装数据的数组"？前者 `char **` 来收，后者用 `int (*p)[N]` 来收。类型对不上时，编译器给的 incompatible pointer type 警告是救命信号，别用强转把它捂住。

写在最后

把这篇压成五条：

1. 步长铁律一句话：`p ± n` 移动 `n × sizeof(所指向的类型)`；多级指针的步长看第一层，与最内层类型无关。
2. 别把"64 位指针占 8 字节"和"步长恒为 8"划等号：`int *p` 步长是 4，`char **p` 步长才是 8。

3. 遇到多级指针先画两层图：上层是指针数组，下层是被指的字符串或对象，箭头落在哪一层，自增就跨哪一层的格子。
4. `argv` 遍历是 `char **` 的标准应用；C 保证 `argv[argc]` 为 `NULL`，拿它当循环终止条件最干净。
5. 二维数组名与 `int **` 类型不兼容，强转只是把编译错误换成运行时崩溃；函数形参请写 `int (*p)[N]`。

下一篇解决一个更隐蔽的困惑：在函数里对指针参数做 `ptr++`，为什么函数返回后，调用者手里的指针纹丝不动？"传址"这个词骗了太多人，传址，仍然是传值。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。